Software Architecture for Agentic AI: Challenges and Way Ahead

Karthik Vaidhyanathan[1] and Henry Muccini[2]

[1] IIIT Hyderabad, India
karthik.vaidhyanathan@iiit.ac.in
[2] University of L'Aquila, Italy
henry.muccini@univaq.it

Abstract. The rapid evolution of Artificial Intelligence (AI), and more recently of Agentic AI, is reshaping the way software-intensive systems are conceived, designed, and operated. While early AI-enabled systems could often be accommodated within established architecting practices by treating AI components as additional modules, the shift toward Agentic AI introduces a more fundamental change: systems are now composed of autonomous agents whose behaviour is inherently non-deterministic, evolves at runtime, and depends on opaque foundation models.
In our prior work, we examined four areas of software architecture that required attention when building machine learning-enabled systems. In this paper, we revisit and extend that analysis for Agentic AI, organising the discussion around the Software Architecture knowledge area of SWEBOK V4.0. Drawing on our experience with two agentic systems, namely, a multi-agent framework for autonomous CloudOps and a multi-agent reasoning framework for self-adaptive systems, we identify open challenges across the four areas: software architecture fundamentals, architecture description, architecting process, and architecture evaluation. We argue that Agentic AI calls for a re-examination of stakeholders, viewpoints, design decisions, and evaluation methods, and we outline short-term and long-term directions for the research and practitioner communities to standardise architecting practices for this emerging class of Agentic AI systems.

Keywords: Software Architecture · Agentic AI · Multi-agent Systems · SWEBOK · Challenges · Research Directions

1 Introduction

Over the recent years, Artificial Intelligence (AI) has transitioned from being yet another capability embedded in selected software components to a central concern that shapes how software systems are designed and implemented. Early integrations of Machine Learning (ML) into software systems retained much of the traditional architecting vocabulary: data pipelines, model components, and inference services were composed alongside conventional modules. In our previous work on architecting ML-enabled systems [1], we examined four areas that

required attention from the architecture community: the architecture framework (stakeholders, concerns, viewpoints), the architecting process (design, analysis, realisation, representation), self-adaptive architecting, and architecture evolution. The advent of Generative AI, and in particular of Agentic AI systems built on top of Large Language Models (LLMs), represents a more disruptive change. An agent in this context is a computational entity that operates autonomously in an environment to pursue defined objectives, perceiving its context and acting upon it [5]. An AI agent extends this notion by incorporating reasoning, learning, and adaptive decision-making, typically with an LLM as its cognitive core [6,11]. An agentic system is then a system composed of multiple such agents, often interacting with tools, humans, and other agents to accomplish tasks that exceed the capability of a single component. There are also emerging works on architectural patterns and frameworks for agent-based software engineering [8,10].

Two characteristics distinguish Agentic AI systems from traditional ML-enabled systems. First, non-determinism is no longer confined to a learned component embedded in a deterministic skeleton; it is embedded in orchestration, planning, and even the structural composition. Second, many decisions now happen at runtime: agents may select tools, spawn new agents, decompose tasks, and coordinate with peers in ways that cannot be fully prescribed at design time. These characteristics challenge the assumptions of traditional architecting practices and motivate a re-examination of the architecture discipline in light of Agentic AI.

To this end, in this work, we revisit our earlier analysis through the structure of the Software Architecture knowledge area of the SWEBOK 4.0 guide [2]. Specifically, we organise the discussion around four sub-areas: (i) software architecture fundamentals, (ii) software architecture description, (iii) the architecting process, and (iv) software architecture evaluation. For each, we summarise what exists, identify the new challenges introduced by Agentic AI, and outline short-term and long-term directions for the community. Our observations are grounded in two case studies drawn from our recent work: MOYA, a multi-agent framework for autonomous CloudOps developed in collaboration with industry [3], and POLARIS, a multi-agent reasoning framework for self-adaptive systems [4]. We also draw on our experience in conducting tutorials and seminars on Agentic AI and Software Architecture [13].

2 Software Architecture and Agentic AI

The emergence of Agentic AI, which builds on a long line of work on multi-agent systems [5] and is shaped by the recent capabilities of LLMs [11], marks another major transition point in the evolution of software-intensive systems. Over the last few decades, software architecture has continuously adapted itself to new computational paradigms ranging from distributed systems and service-oriented architectures to cloud-native systems, cyber-physical systems, and ML-enabled applications [9]. At the same time, AI has evolved from rule-based systems to data-driven machine learning and, more recently, to foundation models capable

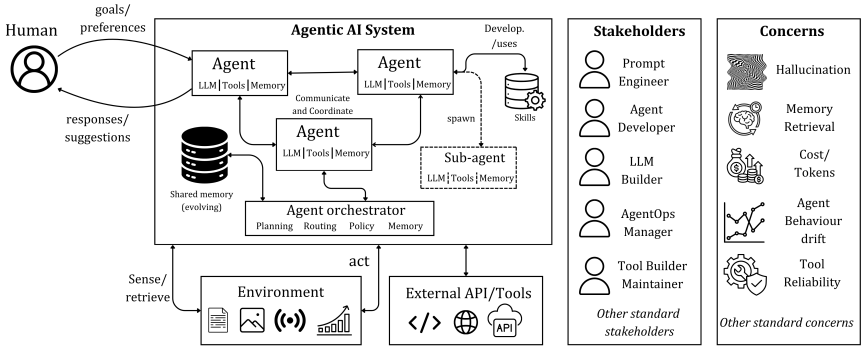


Fig. 1. High-level View of Agentic AI System

of reasoning, generating, planning, and interacting. Agentic AI systems represent the convergence of these trajectories, where AI is no longer treated as an isolated predictive capability embedded into software, but instead becomes an active and adaptive part of the system architecture itself [6].

Unlike earlier ML-enabled systems, where machine learning models often existed as relatively independent components accessed by the surrounding software system, Agentic AI systems dissolve many of these traditional boundaries. Modern agents are not limited to producing predictions or recommendations. They can maintain memory, invoke tools, interact with external services, coordinate with other agents, reason over goals, adapt behavior at runtime, and continuously learn from interactions with both humans and environments [6,10]. Consequently, the architecture of such systems is no longer entirely defined during design time. Instead, parts of the interaction topology, execution flow, and coordination logic emerge dynamically as agents interpret goals, select tools, and negotiate actions during execution.

Recent years have seen growing interest in the use of Generative AI and Agentic AI for software engineering tasks, alongside emerging efforts on applying LLMs and agents to software architecture activities such as design support, architecture recovery, and architecture knowledge management [7,12]. While these works highlight promising opportunities, they largely focus on isolated tasks or application-specific use cases. There remains limited discussion on the broader implications of Agentic AI for the discipline of software architecture itself, particularly concerning emerging architectural challenges, evolving system boundaries, runtime autonomy, and the future directions needed for the software architecture community. This concern is further emphasized by Taibi et al. [28].

Figure 1 illustrates the shift through a high-level view of an Agentic AI system. At the center are multiple interacting agents equipped with foundation models, memory, and tool-use capabilities. These agents communicate and coordinate with one another, access shared memory, invoke external APIs and

tools, retrieve information from the environment, and even spawn specialized sub-agents to accomplish subtasks. Supporting this ecosystem is an orchestration layer responsible for planning, routing, memory coordination, and policy enforcement. The resulting system resembles less a traditional software pipeline and more a continuously evolving socio-technical runtime ecosystem where software logic, AI reasoning, tool interaction, and environmental context are deeply intertwined. This shift also reshapes the role of software architects and the broader engineering ecosystem surrounding such systems. The development of Agentic AI systems requires collaboration across a growing set of stakeholders, including prompt engineers, agent developers, LLM builders, AgentOps managers, and tool maintainers alongside conventional software engineering roles. Furthermore, the concerns that architects must address extend far beyond traditional software quality attributes. Issues such as hallucination, behavioral drift, memory correctness, tool reliability, autonomy boundaries, runtime observability, coordination stability, explainability, and inference cost become first-class architectural concerns. In particular, many of these properties cannot be fully validated statically and instead require continuous runtime monitoring and governance. These characteristics expose a significant gap between existing software architecture practices and the realities of modern Agentic AI systems. In the remainder of the paper, we identify the challenges in architecting Agentic AI systems based on the SWEBOK software architecture areas. We further outline directions for the community to develop better approaches for architecting such systems.

3 Architecting Agentic Systems: Two Case Studies

To ground the discussion in concrete experience, we draw on two Agentic AI systems from practice and research: (i) an open-source framework for autonomous CloudOps, and (ii) a multi-agent reasoning framework for self-adaptive systems. While the first case study is about contributing to the development of Agentic CloudOps, the second is about building an Agentic AI layer on top of existing software systems.

3.1 Case Study A: MOYA, a Multi-Agent Framework for Autonomous CloudOps

MOYA (Meta Orchestrator of Your Agents) is an open source Agentic AI framework developed for autonomous Cloud Operations (CloudOps) [3]. The framework, as described by Parthasarthy et al., was used to build an agentic system that addresses three engineering challenges: i) managing distributed and heterogeneous data sources; ii) maintaining a system whose responsibilities are ever evolving; and iii) supporting extensibility and modularity in the presence of evolving APIs and providers.

MOYA is structured around a Core Engine and a set of agents. The Core Engine includes a Request Handler, an Orchestrator, an LLM Processor, an Agent Selector, an Agent Registry, a Lifecycle Manager, and a Guardrails and

Monitoring component. Agents include a Static Agent for documentation queries (RAG-based), a Summary Agent for hierarchical summarisation of large workloads, a Report Agent that coordinates other agents to produce reports, and several API agents that interact with internal and external services. Agents may be LLM-based or deterministic. An Agent Marketplace and an Agent Portal allow developers and users to register, customise, and deploy agents at runtime.

The case illustrates several points relevant to software architecture. First, the fact that each agent is like a sub-system or a macro-component: the Static Agent is essentially a RAG Agent, which in turn consists of other components like a Vector Database and LLM, while the Report Agent acts as a coordinator over other agents. Second, several decisions that traditional architectures fix at design time, including the selection of an agent for a given request and the registration of new agents, are deferred to runtime (to the orchestrator). Third, the evaluation of the system quality involves both classical software metrics (response time, token usage) and metrics specific to LLM-based components (like BERTScore), agentic components (like tool calling, memory retrieval quality, etc.) and human evaluation across categories such as misclassifications, hallucination, and formatting issues.

3.2 Case Study B: POLARIS, a Multi-Agent Framework for Self-Adaptive Systems

POLARIS is a multi-agent framework for engineering self-adaptive systems [4]. It builds on the three-layer reference model for self-managed systems [14] and proposes an Adaptive Control layer, a Reasoning and Knowledge layer, and a Meta-Learning layer. These layers operate over three nested feedback loops: a Reactive Stabilisation loop for low-latency responses, a Proactive Adaptation loop driven by an LLM-based Reasoner, and a Meta-Learning loop in which a Meta-Learner agent reflects on past experiences and evolves the system's own adaptation policies. POLARIS includes both AI agents (the Reasoner, the Meta-Learner, and the Verifier) and supporting components (the Knowledge Base, the World Model, the Kernel, the Metric Collector, and the Execution Adapter). The Reasoner is configured by a structured prompt that encodes invariants, directives, reasoning workflow, and output format. The Meta-Learner can rewrite parts of this prompt and adjust thresholds at runtime, illustrating a form of architectural adaptation in which the agent's own configuration is part of the design space.

First, it shows how an agentic architecture can implement a classical reference architecture (the three-layer self-managed model) while introducing new forms of non-determinism. Second, it illustrates the need to distinguish between agents, AI agents, and tools as architectural elements with different properties. Third, it demonstrates that the reasoning of an agentic system can be made an explicit object of architectural concern, addressed through a Verifier and through architectural invariants.

Across both cases, we observed that traditional architectural artefacts (component-and-connector diagrams, deployment views, sequence diagrams) remain useful

but are insufficient. They do not capture the prompt configurations, tool signatures, agent registries, and runtime decision policies that significantly shape system behaviour. They also do not naturally express the probabilistic and evolving nature of agent interactions. The lessons learned and the experience gathered while architecting the above systems motivate the discussions going forward.

4 Software Architecture for Agentic AI Systems: The Challenges

This section discusses four areas based on the SWEBOK guide on software architecture that require attention from both the research and practitioner communities when it comes to architecting Agentic AI systems. For each of the areas, we provide details on what are some of the challenges in each of the areas and what needs to be done in future. The challenges have been derived based on our expertise in the field of software architecture, Agentic AI and our experience in architecting different Agentic AI systems (refer to Section 3)

4.1 Software Architecture Fundamentals

There are several elements at the core of the architecture discipline such as the use of the term architecture, stakeholders and concerns, and the uses of architecture [2]. We further discuss these in the light of Agentic AI systems.

Components, Connectors, and Agents: A canonical view of software architecture describes a system as a configuration of components and connectors, with constraints on their composition [16]. In agentic systems, components and connectors continue to exist, but a new class of element appears: the agent. As discussed in Section 3, an agent is more than a component, since it has its own internal architecture, autonomy, and decision-making capability. It is also more than a subsystem, since its behaviour is mediated by an LLM and is therefore non-deterministic by construction. This raises a fundamental question for the discipline: should agents be treated as a distinct architectural element, on a par with components and connectors, or as a specialisation of components with additional properties? Existing work on architectural patterns for foundation-model-based agents [6] suggests the former, while traditional component-based perspectives suggest the latter.

Stakeholders and Concerns: In an agentic system, the set of stakeholders broadens further compared to the ML case. Beyond data scientists, ML engineers, and software developers, the architect must coordinate with prompt engineers, agent developers, and evaluators specialised in LLM behaviour, as well as with domain experts whose knowledge is encoded into agent prompts and tools. In MOYA, for instance, the Agent Marketplace explicitly recognises external developers as stakeholders who deploy and customise agents. The set of concerns also broadens. Classical concerns such as availability, performance, and security remain relevant, but they are joined by concerns that are specific to agentic systems.

These include hallucination control, the cost of inference (expressed in tokens), the risk of unsafe tool use, the explainability of reasoning, and the auditability of agent decisions. The Copenhagen Manifesto on human-centred GenAI for software engineering [15] highlights several of these concerns, including the importance of keeping humans in the loop. The SWEBOK guide itself stresses that concerns evolve over the life cycle of a system and as technologies change [2]; Agentic AI is a clear instance of such an evolution.

Uses of Architecture: SWEBOK identifies several uses of architecture such as providing a shared understanding among stakeholders, supporting analysis and evaluation of alternatives, and enabling reverse engineering of existing systems [2]. In agentic systems, these uses become more important rather than less. The behaviour of a multi-agent system is hard to predict from the inspection of its individual agents, so a shared architectural model is often the only artifact that can support reasoning about emergent behaviour. At the same time, when agents are added or modified at runtime, as in MOYA's Agent Marketplace, the architecture description must remain consistent with a moving target, which raises new challenges that we discuss in Section 4.2.

Our Experience: In our experience of building MOYA and POLARIS, treating agents as a distinct architectural element with explicit notions of cognitive core, tools, memory, and prompt configuration was very important for reasoning about the system. In MOYA, the boundary between a component and an agent was not always sharp: the Static Agent is essentially a RAG service exposed through an agentic interface, while the Report Agent acts as a coordinator over other agents and could equally be described as a connector. In POLARIS, the distinction between AI agents and tools, as defined in [5,6], had to be made explicit to separate the Reasoner and the Meta-Learner (which carry their own goals and reasoning capabilities) from the Knowledge Base and World Model (which support but do not initiate action). The set of concerns we had to manage during architecting was substantially wider than in traditional ML-enabled systems. In MOYA, formatting issues, misclassifications of intent across agents, and hallucination appeared as recurring categories during evaluation; in POLARIS, the cost of LLM inference and the consistency of behaviour across model backends had to be made architecturally visible through explicit invariants and a Verifier.

4.2 Software Architecture Description

Architecture descriptions are the work products that document an architecture for its stakeholders, organised through views and viewpoints, possibly expressed in Architecture Description Languages (ADLs), and constrained by patterns, styles, reference architectures, and frameworks [2]. We discuss how each of these is affected by Agentic AI.

Views and Viewpoints: Classical viewpoints such as the module, component-and-connector, and allocation viewpoints [17,18] remain necessary but not sufficient. They were developed under the assumption that the structures they describe are largely fixed at design time, an assumption that does not hold for agentic systems in which agents, tools, and prompts can change at runtime. This suggests

the need for new viewpoints specific to agentic systems. Candidates include an agent viewpoint that captures the cognitive core, the tools, the memory, and the prompts of each agent; an interaction viewpoint that captures the patterns of communication among agents, including, orchestration (a central agent coordinating others) vs. choreography (agents communicating directly through events or shared memory), synchronous vs. asynchronous, hierarchical vs. peer-to-peer, etc.; a knowledge viewpoint that captures the structure and lifecycle of memory and shared knowledge bases; and a control viewpoint that captures how decisions are distributed between deterministic components, agents, and humans.

Patterns, Styles, and Reference Architectures. SWEBOK lists several established architectural styles, such as layered, pipes and filters, client-server, and publish-subscribe [2]. Recent work has begun to catalogue patterns specific to agentic systems, including patterns for tool use, planning, reflection, and multi-agent collaboration [6]. However, these catalogues are still emerging and do not yet rise to the level of reference architectures comparable to those that exist in domains such as automotive systems, healthcare, or IoT [2]. A reference architecture for agentic systems could capture commonalities such as the separation between cognitive cores and tools, the role of memory and knowledge bases, the placement of guardrails, and the integration of human oversight.

Architecture Description Languages. Wide-spectrum and domain-specific ADLs have been studied for decades [16], but their adoption in industry has remained limited [26]. Agentic AI may renew interest in ADLs for two reasons. First, the complexity of agent interactions and the volume of design decisions to be captured (prompts, tools, memory structures, orchestration policies) make informal box-and-line diagrams insufficient. Second, GenAI itself can lower the barrier to ADL adoption: models can assist in generating ADL specifications from informal descriptions and in supporting analysis on top of those specifications.

Architecture as Significant Decisions. Agentic systems substantially expand the design decision space [2]. New decisions include the choice between LLMs and Smaller Language Models (SLMs); the choice between orchestration and choreography; the choice between using an existing agentic framework and developing a custom platform; the choice between memory structures (short-term, long-term, episodic); the choice of guardrails and verification mechanisms; the choice between existing protocols and emerging agent-specific protocols such as MCP (agent-to-tool) or A2A (agent-to-agent); and the choice of how human oversight is integrated. Many of these decisions are not binary, and they interact strongly with quality attributes such as cost, latency, safety, reliability, etc.

Our Experience. In MOYA, a component-and-connector view captured the Core Engine and its interactions with the Static Agent, Summary Agent, Report Agent, and API agents, but it did not capture the Agent Registry's runtime content, the system prompts that drive each agent, or the tool signatures that define the boundaries of agent autonomy. Documenting MOYA's Agent Marketplace required us to extend our architecture description with what is effectively an agent catalogue, including invocation protocols, authentication tokens, access endpoints, functionality, and permissions. In POLARIS, a deployment view cap-

tured the placement of the Reasoner, Verifier, Meta-Learner, Knowledge Base, and World Model, but it did not capture the prompt configuration that governs the Reasoner's behaviour, nor the meta-learning configuration that allows the Meta-Learner to modify those very prompts at runtime. The pattern across both scenarios were hybrid: MOYA combined a layered organisation (Core Engine over agents) with an orchestration pattern, while POLARIS combined the three-layer reference model for self-managed systems [14] with multi-agent reasoning patterns. We did not find established viewpoints or ADLs that could capture this clearly, and we resorted to a mix of architectural diagrams, prompt templates, and code-level specifications to convey the architecture to collaborators.

4.3 Software Architecture Process

The architecting process can be described around three iterative activities: architecture analysis, synthesis, and evaluation, situated within a broader context that includes architecting in the large [2]. Architecting Agentic AI systems requires revisiting these activities.

Architecture in Context. Agentic systems rarely exist in isolation. They are embedded in enterprise architectures, integrated with existing software systems, and constrained by external factors such as regulatory requirements and the policies of LLM providers. The architect must reason about both the agentic system itself and the system it is embedded in, with their respective concerns, lifecycles, and stakeholders.

Architectural Design. Agentic AI introduces several new questions for architectural design. The first concerns the granularity at which agents should be defined: a few large agents with broad responsibilities, or many small agents with narrow responsibilities. The second concerns the design of architectural tactics for quality attributes that are specific to agentic systems. Tactics for hallucination control, for the bounded use of tools, for the safe interaction with humans (human-in-the-loop or human-on-the-loop or human-on-the-lead), and for the cost-aware selection of LLMs are needed and are only partially documented. Existing work on guardrails for LLMs [22] provides a starting point, but the integration of guardrails as architectural tactics remains underdeveloped. The third concerns the role of GenAI itself in the design process. Recent work shows that GenAI can be used to generate goal models [23] and to support architectural decision making [7]. This suggests an emerging dual relationship in which agents are both an object of architecting and a tool for architecting.

Architecture Analysis. Architecture analysis is described as the activity that gathers and formulates architecturally significant requirements [2]. In an agentic system, this activity is complicated by the inherent non-determinism of agents. Quantitative analysis methods developed for ML-enabled systems, such as quantitative verification of probabilistic models [19,20] and tandem approaches that combine ML with formal analysis [21], are relevant but need to be extended. The challenges include reasoning about the behaviour of multiple agents under

uncertainty, dealing with the lack of formal specifications for LLM behaviour, and acknowledging that the architecture itself can be modified at runtime.

Architecture Realisation and Architecting in the Large. Architecture descriptions historically used text and informal diagrams, and these have been complemented by more rigorous representations [2]. In agentic systems, the realisation of an architecture involves additional artefacts: prompt templates, tool specifications, tool registries, agent registries, and policies for memory management, among others. A practical question is how agile and architecture-up-front approaches interact in this setting. Following a strictly emergent approach is risky because the design space is large and the interactions among agents are hard to anticipate. A purely up-front approach is also problematic because the rapid evolution of LLMs and frameworks invalidates design decisions on short timescales. A pragmatic middle ground is suggested by the work on continuous architecture [24], which advocates a risk-driven and incremental approach.

Architecting in the Large At the scale of an organisation, Agentic AI raises questions about how teams should be organised, how knowledge should be shared, and how reusable assets should be managed. Further, enterprises will have to handle challenges in handling customer-facing agents or third-party agents, and these are areas that require further investigation when it comes to architecting large-scale agentic systems.

Our Experience. In MOYA, the architecting process was strongly shaped by domain-driven design. We identified CloudOps sub-domains (cost, dashboard, notifications, reports, static documentation, summaries) and used them to delineate agent responsibilities. This made the system more maintainable than the initial monolithic RAG implementation, but it left open the question of how to choose granularity for new domains. Design decisions in MOYA were rarely binary: the choice between LLM-based and non-LLM-based agents for the Utility Agent, the choice of guardrail mechanisms, and the choice between using an existing framework (AutoGen, CrewAI, LangGraph) and building a custom platform [3] all required trade-off analysis across cost, latency, accuracy, and integration constraints. However, there was a lack of a systematic approach in making these decisions. In POLARIS, architecture analysis was made partially explicit at runtime through the Verifier, which checks candidate plans against invariants such as bounds on the number of servers. This shifted part of the analysis activity from design time to runtime and required us to identify, document, and operationalise invariants as first-class architectural artifacts. We also experienced the tension between agile and up-front approaches: the choice of LLM, framework, and prompting strategy had to be revisited several times during development as the underlying models evolved.

4.4 Software Architecture Evaluation

Architecture evaluation is the activity that validates whether the chosen solutions satisfy architecturally significant requirements, and discusses it through four sub-areas: “goodness” in architecture, reasoning about architectures, architecture reviews, and architecture metrics [2,25].

"Goodness" in Architecture. The goodness of an architecture is described in terms of its robustness over its lifetime, its fitness for intended use, its feasibility and cost-effectiveness, and its clarity to those who must construct, use, and maintain the software [2]. Each of these criteria takes on a new shape for Agentic AI. Robustness must now account for the non-determinism of LLM-based components and for the runtime evolution of agents (sub-agents that emerge, skills that get generated), their prompts, and their tool sets. Fitness for use must account for failure modes that are specific to agentic systems, including hallucination, unsafe tool use, and the cost of inference. Feasibility and cost-effectiveness depend not only on engineering effort but also on the operational cost of running LLM inference at scale. Clarity is challenged by the fact that significant parts of the architecture, such as prompts and memory structures, are not captured by classical architectural notations. The trade-off analysis at the heart of methods such as ATAM [25] therefore needs to be extended to include trade-offs that are specific to agentic systems, for example, between accuracy and inference cost, or between autonomy and human oversight. Moreover, the existing analysis method needs to be extended to runtime scenarios like sub-agent spawning and skill generation to have a framework for deciding whether to generate a skill or use an existing one or whether it is conducive to generate a sub-agent.

Reasoning about Architectures. Reasoning about an architecture benefits from having robust, existing architecture descriptions to query and analyse [2]. In agentic systems, this benefit is harder to obtain. Significant parts of the architecture, including agent registries, prompt configurations, tool signatures, skill repositories, etc., are not captured by classical viewpoints, as discussed in Section 4.2. Furthermore, the system's behaviour depends on the combined dynamics of multiple non-deterministic agents, which limits the applicability of classical reasoning frameworks. Quantitative analysis methods developed for ML-enabled systems, such as quantitative verification of probabilistic models [19,20] and tandem approaches that combine ML with formal analysis [21], are relevant starting points but need to be extended. A practical implication is that reasoning about agentic systems often has to be partially shifted to runtime, through mechanisms such as runtime verification and online monitoring.

Architecture Reviews. Classical architecture reviews assume that the architecture under review is a relatively stable object, documented through architecture descriptions, and amenable to inspection against a checklist or through structured methods such as ATAM, SAAM, or active reviews [25,2]. In agentic systems, this assumption is weakened. Agents evolve through interactions: their prompts may be modified by humans or by other agents through meta-learning, new tools may be registered, new skills may be added, agents may spawn sub-agents, and the underlying LLM may be replaced. The notion of architectural conformance, which is central to reviews, must therefore be redefined to allow for principled change. Scenario-based reviews remain relevant but must include scenarios specific to agentic systems, including scenarios in which agents hallucinate, misuse tools, or interact unsafely with humans. Reviews should also account for the expanded notion of versioning that agentic systems introduce: the version of an

agent is not only its code but also its prompt, its set of tools, its memory state, and the underlying LLM [27].

Architecture Metrics. Existing architecture metrics, such as coupling, cohesion, and cyclic dependencies [2], remain useful for the deterministic parts of an agentic system. They must be complemented by metrics that capture the specific properties of agentic components, including hallucination rate, tool-use accuracy, cost of inference (expressed for example in tokens), latency of reasoning, and degree of human intervention required. The latter is particularly important in safety-critical or compliance-critical settings, where the level of autonomy of the system is itself a quality attribute. The integration of such metrics with classical architecture metrics into a unified view is an open research problem, as is their connection to continuous-deployment metrics such as deployment frequency and mean time to restore service [2].

Our Experience. Evaluating MOYA required us to combine automated NLP metrics (ROUGE, BLEU, METEOR, BERTScore) with human evaluation conducted by a team of product managers and platform engineers, who categorised issues into misclassification, hallucination, and formatting [3]. This combination revealed properties that no single method could surface: the automated metrics quantified semantic alignment with gold-standard responses, while the human evaluation identified categories of failure that required targeted prompt engineering. The efficiency dimension required a separate evaluation strategy based on response time and token usage, neither of which is part of the classical architecture metrics catalogue. In POLARIS, we evaluated the framework across two self-adaptive exemplars (SWIM and SWITCH) and across multiple LLM backends, observing that the architectural design (Reasoner, Verifier, Meta-Learner, Knowledge Base, World Model) was more consistent across runs than the individual LLM choice would predict [4]. We took this as evidence that architectural guardrails can mitigate the variability of LLM-based components, but we also observed that establishing this claim required extensive empirical evaluation and substantial overhead in terms of inference latency and token cost. In both systems, the question of architectural conformance was effectively replaced by continuous evaluation: we accepted that the architecture would evolve through prompt updates and threshold tuning, and we treated evaluation as an ongoing activity rather than a milestone.

5 Discussion and Way Ahead

The four areas discussed in Section 4 are not independent. In this section, we consolidate our experiences based on the cases into a set of lessons learned, and we synthesise the directions identified across the four sub-areas into a small number of short-term and long-term research priorities for the community.

5.1 Short-term Directions

In the near term, we see four priorities that the architecture community can begin to address with the methods and case studies that already exist.

Vocabulary and First-Class Concepts. The notion of agent should be elevated to a first-class architectural element, with a clear definition that distinguishes it from a component, a connector, and a subsystem, and that captures its internal anatomy as part of the architecture rather than as an implementation detail. The set of stakeholder roles and concerns specific to agentic systems should be catalogued to create a common vocabulary.

Viewpoints and Description. There is a need to define new architecture viewpoints that consider the different runtime dimensions of Agentic AI systems. Some candidate viewpoints include an agent viewpoint (cognitive core, tools, memory, prompts), an interaction viewpoint (orchestration vs. choreography), a knowledge viewpoint (memory and shared knowledge), and a control viewpoint (decision distribution between humans, agents, and deterministic components). A pragmatic direction for description languages is to extend existing ADLs and modelling languages such as UML, SysML, C4 Modeling with constructs for agents, prompts, and tools, rather than develop new languages from scratch.

Decision Support and Tactics. Methods are needed to support architectural design decisions specific to agentic systems, in particular the choice of agent granularity, the choice between orchestration and choreography, and the integration of guardrails. These decisions interact with multiple quality attributes and should be supported by trade-off analysis techniques in the spirit of ATAM [25], extended to account for non-deterministic behaviour and runtime evolution. Tactics for hallucination control, cost-aware reasoning, reliable skill generation, subagent spawning, and human oversight should be documented and validated.

Metrics and Continuous Evaluation. A catalogue of metrics specific to agentic systems should be developed, capturing properties such as hallucination rate, tool-use accuracy, cost of inference, autonomy level, etc., and integrated with classical architecture metrics into a unified view. Evaluation should be recognised as a continuous, multi-stakeholder activity rather than a discrete milestone.

5.2 Long-term Directions

In the longer term, we see three directions that require more sustained effort.
Reference Architectures for Recurring Classes of Agentic Systems. Reference architectures, comparable to those that exist in domains such as automotive systems, healthcare, and IoT, should be developed for recurring classes of agentic systems. Promising candidates include autonomous operations (as exemplified by MOYA), self-adaptive systems (as exemplified by POLARIS), and human-in-the-loop decision support. The agent design pattern catalogues that are beginning to emerge [6] provide a useful foundation but do not yet rise to the level of reference architectures. Such reference architectures can also serve as an effective mechanism to support controlled autonomy of systems at runtime.

Hybrid Analysis Methods for Non-deterministic and Evolving Architectures. Architecture analysis methods that combine offline reasoning (formal verification of invariants, scenario-based analysis) with online assurance (runtime verification,

monitoring, and human review) deserve sustained investigation. Runtime verification and probabilistic model checking [19] are natural starting points, but they need to be extended to systems whose architecture itself evolves continuously.

The Dual Role of GenAI in Architecting. The same technology that we are learning to architect is also a tool that can support the architecting process itself. Recent work shows that GenAI can be used to generate goal models for self-adaptive systems [23] and to support architectural decision making [7]. This dual role has implications for the skills required of future architects and for the methods that the community needs to develop. It also raises questions of trust and verifiability of AI-assisted architectural artefacts that are likely to occupy the community for some time.

We do not claim that these directions are exhaustive. The challenges, lessons, and directions discussed in this paper are derived from our experience in architecting two agentic systems as well as based on our tutorials [13]. Other concerns, such as technical debt in agentic systems, sustainability, and the management of training and inference data, deserve dedicated attention and were not covered in detail in this work. Moreover, Agentic AI is evolving rapidly. The frameworks, models, and patterns that we observed during the development of MOYA and POLARIS may be superseded with time. However, we have tried to phrase the lessons and directions at a level of abstraction that is robust to such change, while acknowledging that specific observations tied to current LLMs, frameworks, or protocols may evolve over time.

6 Conclusion

In this paper, we have revisited the challenges of architecting AI-enabled systems through the lens of Agentic AI. Building on our prior work on architecting ML-enabled systems and following SWEBOK 4.0, we have identified open challenges across four sub-areas: fundamentals, description, process, and evaluation. We have grounded our analysis in two case studies, MOYA for autonomous CloudOps and POLARIS for self-adaptive systems, that illustrate the architectural characteristics of agentic systems in practice. Our central observation is that Agentic AI is not merely an incremental change to ML-enabled systems but a qualitative shift in which non-determinism, runtime evolution, and hybrid deterministic and non-deterministic composition become central architectural concerns. Addressing this shift requires more than the adaptation of existing methods: it requires a sustained effort by the research and practitioner communities. We hope that the discussions presented here, together with the case studies that ground them, can serve as a useful organising framework for that effort.

Acknowledgments

The authors used generative AI tools to assist with writing this paper. All technical ideas, analyses, results, and conclusions in this paper were conceived, de-

veloped, and verified solely by the authors. The authors take full responsibility for the content of the final manuscript.

References

1. Muccini, H., Vaidhyanathan, K.: Software Architecture for ML-based Systems: What Exists and What Lies Ahead. In: ICSE Workshop on AI Engineering: Software Engineering for AI (WAIN). IEEE (2021)
2. IEEE Computer Society: SWEBOK Guide V4.0. Chapter 2: Software Architecture (2024)
3. Parthasarathy, K., Vaidhyanathan, K., Dhar, R., Krishnamachari, V., Kakran, A., Akshathala, S., Arun, S., Karan, A., Muhammed, B., Dubey, S. and Veerubhotla, M., 2025, April. Engineering llm powered multi-agent framework for autonomous cloudops. In 2025 IEEE/ACM 4th International Conference on AI Engineering–Software Engineering for AI (CAIN) (pp. 201-211). IEEE.
4. Pandey, D., Gupta, V., Singhal, P., Vaidhyanathan, K.: POLARIS: Is Multi-Agentic Reasoning the Next Wave in Engineering Self-Adaptive Systems?. SEAMS 2026
5. Wooldridge, M.: An Introduction to MultiAgent Systems. John Wiley & Sons (2009)
6. Liu, Y., Lo, S.K., Lu, Q., Zhu, L., Zhao, D., Xu, X., Harrer, S., Whittle, J.: Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents. Journal of Systems and Software 220, 112278 (2025)
7. Esposito, M., Li, X., Moreschini, S., Ahmad, N., Cerny, T., Vaidhyanathan, K., Lenarduzzi, V. and Taibi, D., 2025. Generative AI for software architecture. Applications, challenges, and future directions. Journal of Systems and Software, p.112607.
8. He, J., Treude, C., Lo, D.: LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision, and the Road Ahead. ACM Transactions on Software Engineering and Methodology 34(5), 1–30 (2025)
9. Wan, Z., Zhang, Y., Xia, X., Jiang, Y. and Lo, D., 2023, November. Software architecture in practice: Challenges and opportunities. In Proceedings of the 31st ACM joint European software engineering conference and symposium on the foundations of software engineering (pp. 1457-1469).
10. Wang, Y., Zhong, W., Huang, Y., Shi, E., Yang, M., Chen, J., Li, H., Ma, Y., Wang, Q., Zheng, Z.: Agents in software engineering: Survey, landscape, and vision. Automated Software Engineering 32(2), 1–36 (2025)
11. Xi, Z., Chen, W., Guo, X., He, W., Ding, Y., Hong, B., Zhang, M., Wang, J., Jin, S., Zhou, E., et al.: The Rise and Potential of Large Language Model-Based Agents: A Survey. Science China Information Sciences 68(2), 121101 (2025)
12. Zhang, Y., Li, R., Liang, P., Sun, W. and Liu, Y., 2025, June. Knowledge-based multi-agent framework for automated software architecture design. In Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (pp. 530-534).
13. Vaidhyanathan, K. and Muccini, H., 2025, September. Software Architecture in the Age of Agentic AI. In European Conference on Software Architecture (pp. 41-49).
14. Kramer, J., Magee, J.: Self-Managed Systems: an Architectural Challenge. In: Future of Software Engineering (FOSE). IEEE Computer Society, pp. 259–268 (2007)

15. Russo, D., Baltes, S., van Berkel, N., Avgeriou, P., Calefato, F., Cabrero-Daniel, B., Catolino, G., Cito, J., Ernst, N., Fritz, T., et al.: Generative AI in software engineering must be human-centered: The Copenhagen Manifesto. Journal of Systems and Software 216, 112115 (2024)
16. Taylor, R.N., Medvidović, N., Dashofy, E.: Software Architecture: Foundations, Theory, and Practice. Wiley (2009)
17. Kruchten, P.B.: The 4+1 View Model of Architecture. IEEE Software 12(6), 42–50 (1995)
18. Clements, P., Bachmann, F., Bass, L., Garlan, D., Ivers, J., Little, R., Merson, P., Nord, R., Stafford, J.: Documenting Software Architectures: Views and Beyond, 2nd edn. Addison-Wesley (2011)
19. Calinescu, R., Ghezzi, C., Kwiatkowska, M., Mirandola, R.: Self-Adaptive Software Needs Quantitative Verification at Runtime. Communications of the ACM 55(9), 69–77 (2012)
20. Casimiro, M., Soares, D., Garlan, D., Rodrigues, L., Romano, P.: Self-adapting Machine Learning-based Systems via a Probabilistic Model Checking Framework. ACM Transactions on Autonomous and Adaptive Systems 19(3), Article 18 (2024)
21. Cámara, J., Muccini, H., Vaidhyanathan, K.: Quantitative Verification-Aided Machine Learning: A Tandem Approach for Architecting Self-Adaptive IoT Systems. In: IEEE International Conference on Software Architecture (ICSA). IEEE (2020)
22. Dong, Y., Mu, R., Jin, G., Qi, Y., Hu, J., Zhao, X., Meng, J., Ruan, W., Huang, X.: Building Guardrails for Large Language Models. arXiv preprint (2024)
23. Nakagawa, H., Honiden, S.: MAPE-K Loop-based Goal Model Generation Using Generative AI. In: IEEE 31st International Requirements Engineering Conference Workshops (REW). IEEE (2023)
24. Erder, M., Pureur, P., Woods, E.: Continuous Architecture in Practice: Software Architecture in the Age of Agility and DevOps. Addison-Wesley (2021)
25. Clements, P., Kazman, R., Klein, M.: Evaluating Software Architectures. Addison-Wesley (2001)
26. Malavolta, I., Lago, P., Muccini, H., Pelliccione, P. and Tang, A., 2012. What industry needs from architectural languages: A survey. IEEE Transactions on Software Engineering, 39(6), pp.869-891.
27. Biswas, S., Bhatt, H. and Vaidhyanathan, K., 2026. Architecting AgentOps Needs CHANGE. In Proceedings of 5th International Conference on AI Engineering (CAIN) 2026.
28. Taibi, D., Muccini, H., Vaidhyanathan, K., Kalinowski, M., Albano, M., Alves, A.P.S., Cerqueira, R., Devino, M., Esposito, M., Falcão, R. and Henning, V., 2026. A Research Agenda on Agents and Software Engineering: Outcomes from the Rio A2SE Seminar. arXiv preprint arXiv:2605.11720.